

Sorting Algorithms Analysis

From: Kirill Grichanichenko

Date: December 01, 2024

Subject: Comparing Sorting Algorithms

Introduction

This report investigates six sorting algorithms and compares their performances against each other. The report includes Bubble sort, Insertion sort, Merge Sort, Iterative Merge sort, Shell sort, and Quick sort. Each algorithm was implemented using C++ on Visual Studio Code and tracked using microseconds while sorting random integer values of sizes 10, 100, 1000, 5000, 10000, 15000, 25000, 30000, 50,000, 75,000 and 100,000 integers. To further test the non-square sort algorithms, further integer values sizes were added of 250,000. During the experiment, for each size, an algorithm ran five trials, and the average result of the trials was graphed.

Table 1: Algorithm Time Complexities

Algorithms	Best Case (Ω)	Average Case (Θ)	Worst Case (O)
Bubble Sort	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$
Iterative Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$
Shell Sort	$\Omega(n \log n)$	$\Theta(n^{3/2})$	$O(n^2)$
Quick Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$

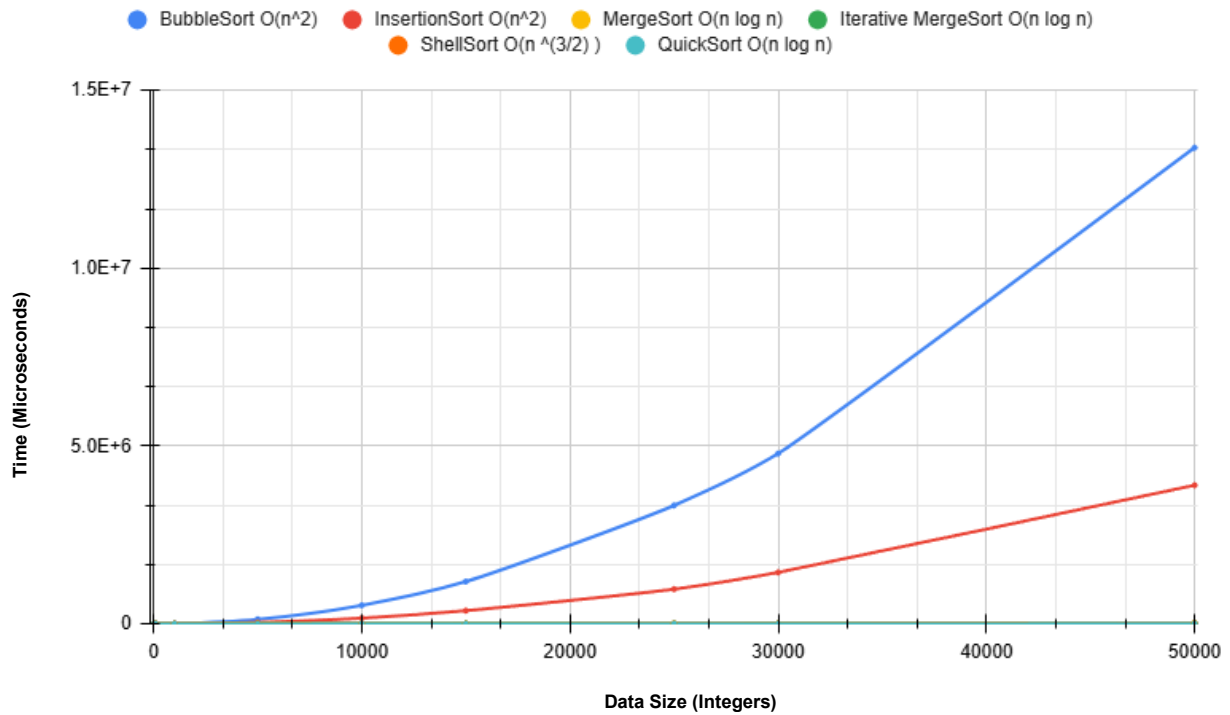
Data and Analysis

This section covers the analysis of different sorting algorithms based on the graphs.

All Sorting Algorithms

Figure 1 shows the comparison of each sorting algorithm on one graph. We can see that the insertion sort and bubble sort both have an average time complexity of $O(n^2)$, and both significantly stray away from the other non-square sort algorithms, making it difficult to see the differences between the other algorithms. We can visually see that with 1000 integer values, the bubble sort strays from the other algorithms in time and takes (6501.6 μ s) to sort. At around 10000 integers, we can visually see Insertion sort stray from the other algorithms and takes (161066.6 μ s) to sort. Taking a random non-square algorithm like Merge sort, we can see it takes (473 μ s) to sort 1000 integers and (4891.4 μ s) to sort 10000 integers, showing us the huge deviation between the square and non-square algorithms.

Figure 1: Comparison of All Sorting Algorithms

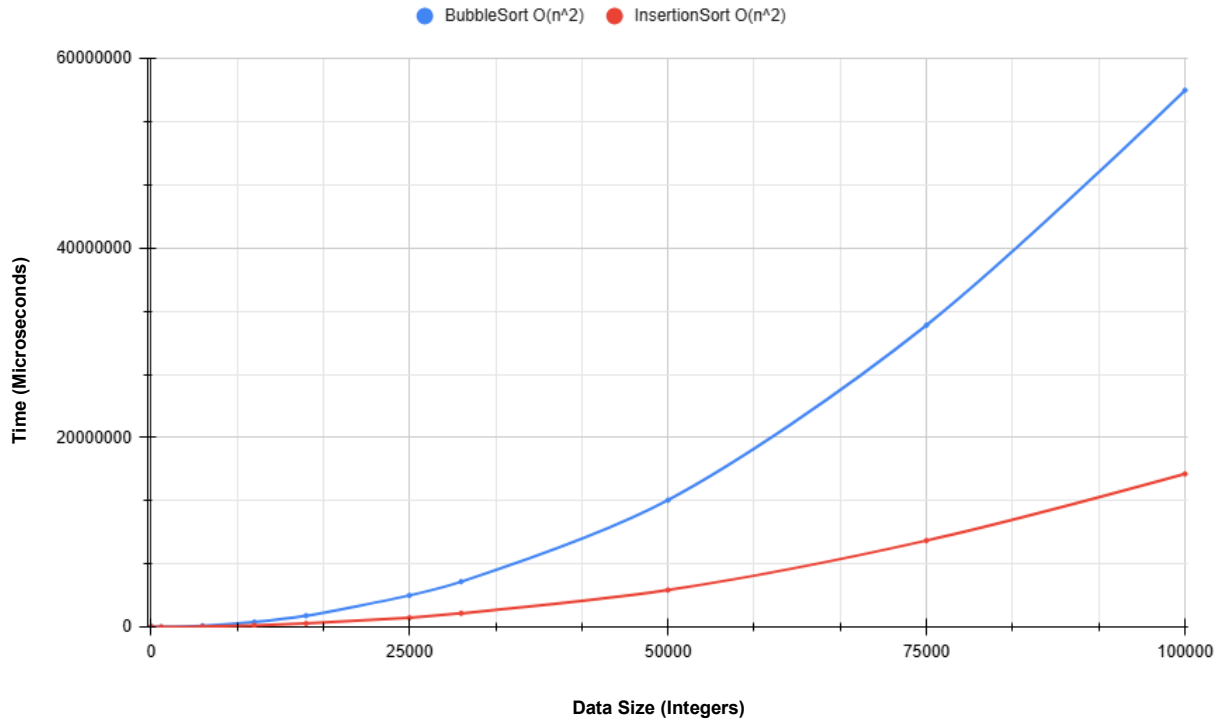


Square Sorting Algorithms

Figure 2 shows us the comparison of just the Insertion and Bubble sort algorithms. We can see both sorts have a relatively similar growth rate between data ranges of 10 to 10000 integers. We can see Bubble sort taking (522692.4 μ s.) and Insertion sort taking (161066.6 μ s.) to sort 10000 integers, giving us the highest difference of (1087374.2 μ s.) between 10 to 10000 integers. However, passing 10000 integers, we can see that the bubble sort becomes notably slower and takes about 3.5 times as long as the insertion sort with 100,000 integers. With 100,000 integers we can see the highest time difference between the sorts. Calculating the difference with 100,000 integers, Bubble sort takes on average (56664886.8 μ s.) and Insertion sort takes on average (16165190.1 μ s.) to sort 100,000 integers, giving us a time difference of (404994696.7 μ s.). Based on this, we can see that between the Bubble sort and Insertion sort, the slowest to fastest sorting algorithms are:

- Bubble sort (56664886.8 μ s. to sort 100,000 integers)
- Insertion sort (16165190.1 μ s. to sort 100,000 integers)

Figure 2: Comparison of Square Sorting Algorithms



Non-Square Sorting Algorithms

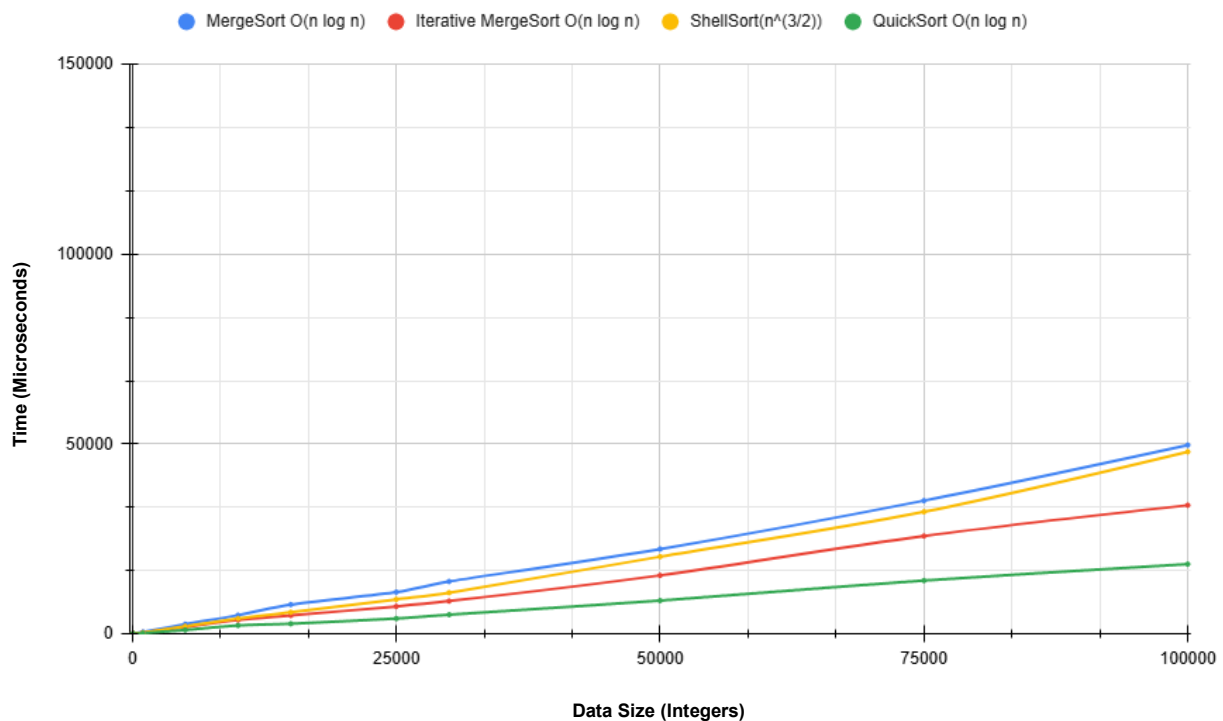
To analyze the rest of the sorting algorithms, Figure 3 compares all the sorting algorithms without the Bubble and Insertion sort. Analyzing the data from Figure 3, if we look at the time to sort the largest data set of 100,000 integers, we can see the slowest to fastest sorting algorithms. The slowest to fastest sorting algorithms are:

- Merge sort (49718.8 μ s.)
- Shell sort (47946.8 μ s.)
- Iterative Merge sort (33867.4 μ s.)
- Quick sort (18350 μ s.).

We can see that around 10 to 5000 integers all the sorting algorithms take relatively the same time, with the biggest deviation being (1529 μ s.) at 5000 integers between Merge sort (2532.6 μ s.) and Quick sort (1003.6 μ s.). We can see the Iterative Merge sort and Shell sort perform about the same between 10 and 15000 integers with the Iterative Merge sort (4831.8 μ s. at 15000 integers) staying a step faster than the Shell sort (5667.8 μ s. at 15000 integers). Following the Shell sort passed 15000 integers, and we can see that the deviation between the Iterative Merge sort becomes larger. With Shell sort taking (47946.8 μ s.) and Iterative Merge sort taking (33867.4 μ s.) to sort 100,000 integers, giving us a difference of (14079.4 μ s.). However, the deviation between the Shell sort and regular Merge sort becomes smaller. With Shell sort taking

(47946.8 μ s.) and regular Merge sort taking (49718.8 μ s.) to sort 100,000 integers, giving us a difference of (1772 μ s.). This shows that with larger data, the Merge sort could be more efficient than the Shell sort. This makes sense because the Merge sort has an average time complexity of $O(n \log n)$, and the Shell sort has an average time complexity of $O(n^{3/2})$. Looking at the Quick sort, we can see that it performs significantly faster than all the other sorts right from the start and deviates farther from them as the data increases. Quick sort increased in time at about (15000 μ s.) from 10 to 50000 integers, which is fairly small compared to Merge sorts increase of about (45000 μ s.) from 10 to 50000 integers.

Figure 3: Comparison of Non-square Sorting Algorithms

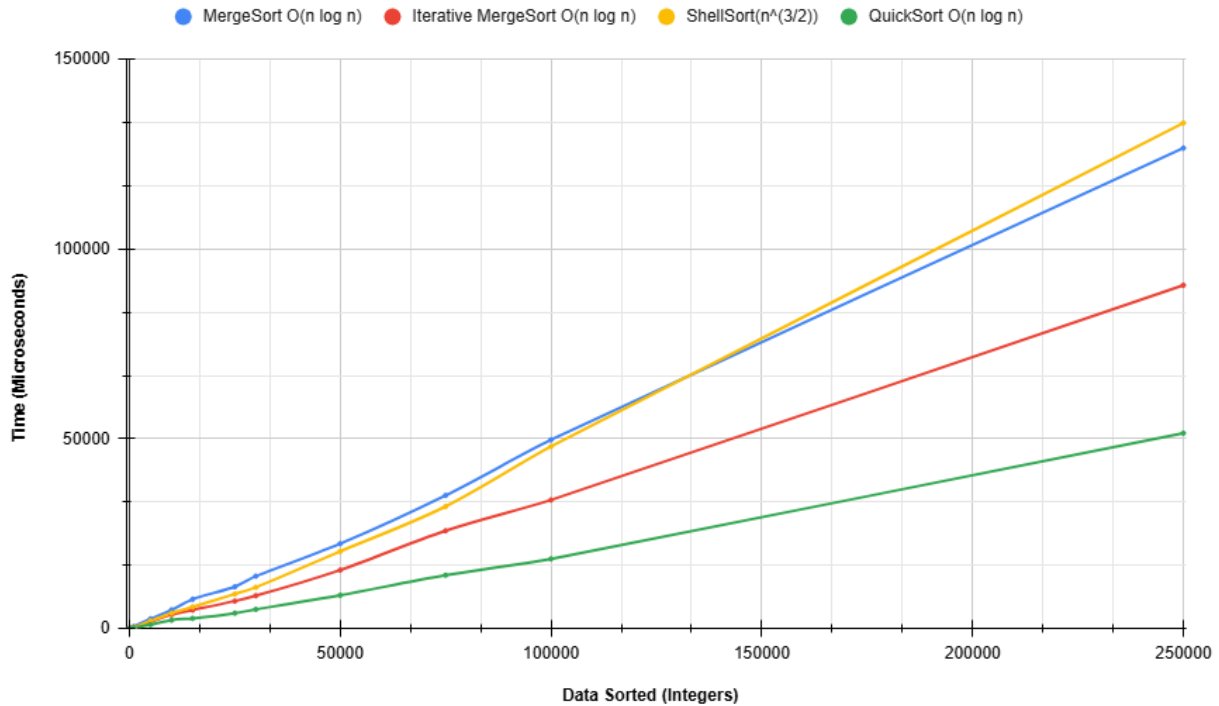


Non-Square Sorting Algorithms with Large Data Sets

Following the theory that at some large data set Merge sort will start performing better than the Shell sort, Figure 4 shows us a comparison of all non-square sorting algorithms with large data sets, up to 250,000 integers. We can see Iterative Merge sort and Quick sort still perform at the same rate as with lower data sets, so there is nothing new here. However, looking at Merge sort and Shell sort, we can see that passed 100,000 integers the Shell sort becomes slower and slower until finally at around (135,000 integers) the Merge sort overcomes the Shell sort and starts sorting faster. With even larger data sets, we can clearly see the Merge sort performing more efficiently than Shell sort. Looking at 250,000 integers, we can see that Merge sort takes (126640 μ s.) and the Shell sort takes (133250.6 μ s.) to sort the integers. This confirms the theory that with

larger data sets, the Merge sort performs more efficiently than the Shell sort; especially data sets past (135,000 integers).

Figure 4: Comparison of Non-square Sorting Algorithms with Large Data Sets



Conclusion

Combining and analyzing all the sorts from the graphs with small data sets, we can see the slowest to fastest sorting algorithms by comparing the time to sort (100,000 integers). The slowest to fastest sorting algorithms with small data sets are:

- Bubble sort (56664886.8 μ s.)
- Insertion sort (16165190.1 μ s.)
- Merge sort (49718.8 μ s.)
- Shell sort (47946.8 μ s.)
- Iterative Merge sort (33867.4 μ s.)
- Quick sort (18350 μ s.).

However, by analyzing all the sorts from the graph with large data sets, we can see the slowest to fastest sorting algorithms by comparing the time to sort (250,000 integers). The slowest to fastest sorting algorithms with large data sets are:

- Bubble sort (56664886.8 μ s. At 100,000 integers)

- Insertion sort (16165190.1 μ s. At 100,000 integers)
- Shell sort (133250.6 μ s.)
- Merge sort (126640 μ s.)
- Iterative Merge sort (90476.2 μ s.)
- Quick sort (51478 μ s.)

Analyzing the $O(n^2)$ algorithms, Insertion sort performs more efficiently than Bubble sort because it has a best case of $O(n)$ for partially sorted lists, but Bubble sort's best case is $O(n^2)$. Analyzing Merge, iterative Merge, Shell, and Quick sorts, we can see that all the sorts have an average case of $O(n \log n)$ except the Shell sort which has an average case of $O(n^{3/2})$. Quick sort and Iterative Merge sorts perform the best because they have a worst case of $O(n \log n)$, but the quick sort is slightly faster since it does not use an additional copy of data structures. The next best are Shell and Merge sorts, which have different average cases of $O(n \log n)$ for Merge sort and $O(n^{3/2})$ for shell sort. Despite these differences, the graphs show Shell sort performs more efficiently than Merge sort with data sets less than (135,000 integers). In theory Merge sort should perform faster than the Shell sort because Merge sort has a worst case of $O(n \log n)$ and Shell sort has a worst case of $O(n^2)$, but the reason that Shell sort performs faster could be due to it having faster operation with lower data. We can see in Figure 2 that as the data grew, the deviation between Shell sort and Merge sort became smaller, predicting that at some larger data size, the Merge sort could become more efficient. This has been then proven in figure 4, where we can see the Merge sort overtook in efficiency the Shell sort and performed faster sorts with large data sets passed (135,000 integers).

Appendix: Graph Data

10	3	1.6	4.8	3.2	2.2	2.8
100	62.4	22.4	40.4	22.4	18.6	15
1000	6501.6	2265.2	473	282.2	280.6	170.8
5000	128982	42001.6	2532.6	1763.2	1983.2	1003.6
10000	522692.4	161066.6	4891.4	3621.8	4029.4	2193.6
15000	1188843.8	372141	7696.4	4831.8	5667.8	2648.2
25000	3329416.6	975911.2	10984	7212.8	9068	4023.2
30000	4786909.6	1446746	13779.6	8625.8	10828.4	5034
50000	13388682.2	3897647.6	22340.6	15386.8	20309.2	8747.4
75000	31856610.2	9123988.4	35077.2	25751	32144.4	14028.4
100000	56664886.8	16165190.1	49718.8	33867.4	47946.8	18350
250000			126640	90476.2	133250.6	51478
Data	BubbleSort	InsertionSort	MergeSort	Iterative MergeS	ShellSort	QuickSort
BubbleSort	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Average
10	2	2	6	3	2	3
100	62	63	62	62	63	62.4
1000	6412	6698	6490	6581	6327	6501.6
5000	129102	127963	131313	127376	129156	128982
10000	524354	525724	520961	518707	523716	522692.4
15000	1184112	1195372	1188938	1189450	1186347	1188843.8
25000	3337100	3324134	3331400	3329397	3325052	3329416.6
30000	4782521	4796086	4792365	4793098	4770478	4786909.6
50000	13387951	13330356	13397232	13395725	13432147	13388682.2
75000	31276475	32043756	31740374	31782416	32440030	31856610.2
100000	56585634	56098848	56033296	56855290	57751366	56664886.8
250000						
InsertionSort	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Average
10	2	2	1	1	2	1.6
100	22	23	22	23	22	22.4
1000	1859	2326	2833	2079	2229	2265.2
5000	41927	41240	41394	44318	41129	42001.6
10000	161318	160244	162738	154563	166470	161066.6
15000	378352	367380	374961	381942	358070	372141
25000	977507	980584	973180	976722	971563	975911.2
30000	1442075	1432316	1447433	1458586	1453320	1446746
50000	3897322	3910413	3898386	3884475	3897642	3897647.6
75000	9080842	9222292	9139131	8873920	9303757	9123988.4
100000	16194288	16226666	16219978	15662996	16522025	16165190.1

MergeSort	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Average
10	5	4	5	5	5	4.8
100	41	40	40	40	41	40.4
1000	465	469	481	467	483	473
5000	2679	2883	2755	2147	2199	2532.6
10000	5207	3907	5747	4861	4735	4891.4
15000	7442	7731	8710	7459	7140	7696.4
25000	10682	11160	11085	10851	11142	10984
30000	14892	13429	13641	13460	13476	13779.6
50000	22322	22321	22191	22324	22545	22340.6
75000	34581	34639	35892	35361	34913	35077.2
100000	49775	55644	47991	47947	47237	49718.8
250000	126089	126353	128307	126952	125499	126640

Iterative MergeSort	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Average
10	3	3	3	3	4	3.2
100	23	22	22	23	22	22.4
1000	277	293	277	277	287	282.2
5000	1737	1934	1737	1607	1801	1763.2
10000	3965	3834	3462	3004	3844	3621.8
15000	4027	5579	4407	5703	4443	4831.8
25000	7228	6992	7128	7270	7446	7212.8
30000	8662	8671	8664	8683	8449	8625.8
50000	15321	15271	15506	15651	15185	15386.8
75000	27369	24701	26021	25224	25440	25751
100000	34346	33951	33011	35578	32451	33867.4
250000	88973	87609	91938	92651	91210	90476.2

ShellSort	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Average
10	2	2	2	3	2	2.2
100	20	18	18	18	19	18.6
1000	278	285	286	276	278	280.6
5000	1983	1978	1977	2007	1971	1983.2
10000	4380	4389	3226	4493	3659	4029.4
15000	6023	5952	4734	4789	6841	5667.8
25000	9080	9014	9139	9147	8960	9068
30000	10980	10763	10729	10763	10907	10828.4
50000	20357	20296	20209	20447	20237	20309.2
75000	31988	31571	33031	32827	31305	32144.4
100000	47750	48853	49748	46257	47126	47946.8
250000	140686	134840	128685	132177	129865	133250.6

QuickSort	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Average
10	2	3	3	3	3	2.8
100	15	16	14	14	16	15
1000	170	171	171	171	171	170.8
5000	1003	1008	1003	999	1005	1003.6
10000	2147	2225	2169	2158	2269	2193.6
15000	2338	3353	2597	2286	2667	2648.2
25000	4001	3974	4167	3988	3986	4023.2
30000	5177	4919	5040	4938	5096	5034
50000	9109	8841	8695	8479	8613	8747.4
75000	14175	14439	13831	14098	13599	14028.4
100000	18286	17982	17908	18920	18654	18350
250000	50282	53422	51756	51736	50194	51478